Workload-Aware Adaptive Page Replacement

Name:Arpit Raj

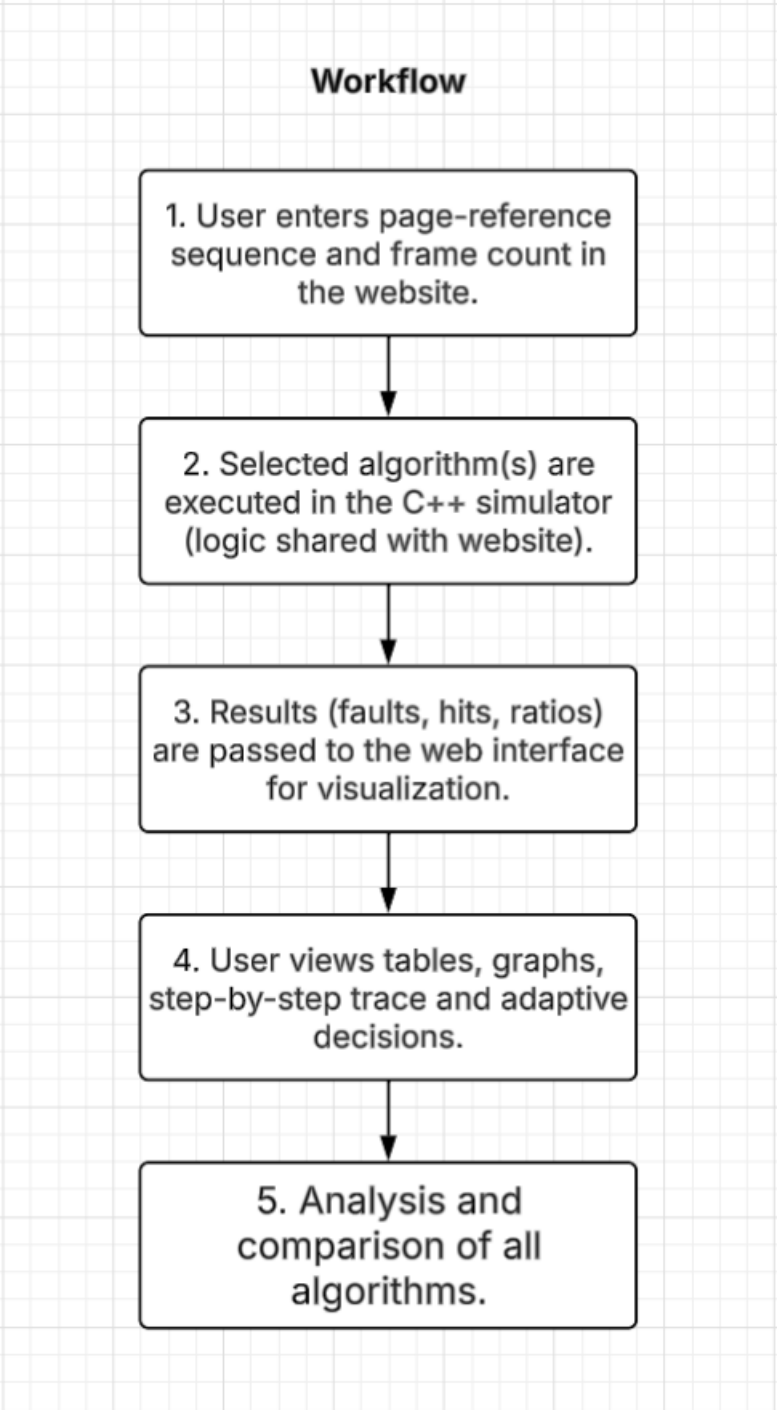
Reg.No:24BCE2897

1. Project Overview

Workload-Aware Adaptive Page Replacement is an Operating Systems project that demonstrates how page replacement algorithms manage memory when the number of pages requested by a process exceeds the available physical memory frames. When a required page is not present in memory, a page fault occurs, and the operating system must decide which existing page to replace.

This project implements and compares four page replacement algorithms: First-In, First-Out (FIFO), Least Recently Used (LRU), Optimal, and Adaptive Page Replacement. It includes a C++ simulator and a standalone web-based simulator that displays algorithm performance through tables, graphs, and step-by-step execution traces.

2. Workflow



3. Implementation Details

The project is implemented using two components: a C++ command-line simulator and a web-based visualization interface.

C++ Simulator: The C++ program implements FIFO, LRU, Optimal, and Adaptive page replacement algorithms. It accepts a page-reference sequence and the

number of available frames, simulates page accesses, and calculates page faults, page hits, hit ratio, and fault ratio.

Web Interface: The website is developed using HTML, CSS, and JavaScript. HTML defines the structure, CSS provides the layout and styling, and JavaScript implements the simulation logic, calculates performance metrics, and displays the results through tables, graphs, and step-by-step execution traces.

Adaptive Algorithm: The Adaptive approach maintains shadow simulations for FIFO and LRU and compares their page-fault counts over a recent window of up to eight references, including the current reference. It selects the policy with fewer faults and chooses FIFO when both have equal fault counts. The selected policy determines which page is replaced when necessary.

Input and Output: The user can provide a custom page-reference sequence or select a supported workload pattern and specify the number of frames. The program then displays the performance metrics and replacement behavior for comparison.

Execution: The C++ simulator is compiled using a compatible C++ compiler and executed through the terminal in VS Code. The web interface runs locally in a web browser by opening the index.html file.

The two components provide complementary ways to execute, analyze, and demonstrate page replacement algorithms.

4. Technology Stack and Configuration

The project uses the following technologies:

- C++: Used to implement the command-line page replacement simulator.
- HTML: Defines the structure and components of the website.
- CSS: Provides the website layout, dark theme, colors, tables, and responsive styling.
- JavaScript: Implements the website simulations, calculates performance metrics, and updates the results and graphs.
- Web browser: Used to run the website locally.
- GitHub: Used to store and share the project source code and documentation.

The website consists of three files: index.html, style.css, and script.js. It can be run locally by opening index.html in a web browser. The website does not require a database, login system, or additional framework.

The C++ simulator requires a compatible C++ compiler. The exact compiler version and compilation command should be documented according to the development environment used.

5. Algorithms Implemented

FIFO (First-In, First-Out): Replaces the page that has been in memory for the longest time. It is simple to implement but may replace a page that will soon be needed again.

LRU (Least Recently Used): Replaces the page that has not been accessed for the longest time. It uses recent access history to make replacement decisions.

Optimal Page Replacement: Replaces the page whose next use is farthest in the future. It is primarily used as a theoretical benchmark because it requires knowledge of future page references.

Adaptive Page Replacement: The website uses two hidden shadow memories to track FIFO and LRU performance over a recent window of up to eight references, including the current reference. When a replacement is needed, it selects the strategy with fewer faults in that window. If both strategies have equal faults, FIFO is selected. This approach uses observed references rather than future information. Adaptive is a heuristic and is not guaranteed to outperform FIFO or LRU on every workload.

The C++ and website implementations should be documented separately if their Adaptive decision logic differs.

6. Website Features and Implementation

The website provides a simple dashboard for comparing all four algorithms using the same page-reference sequence and number of frames.

Its main features include:

- Input fields for the page-reference sequence and number of frames.
- A simulation button to execute the algorithms.
- A results table displaying page faults, page hits, hit ratio, and fault ratio.

- A bar chart comparing page faults across algorithms.
- A step-by-step simulation table showing frame contents and whether each reference results in a hit or fault.
- An Adaptive decision table showing the selected strategy and replacement information.

These features make it easier to observe how different replacement strategies behave under different workloads.

7. Testing and Validation

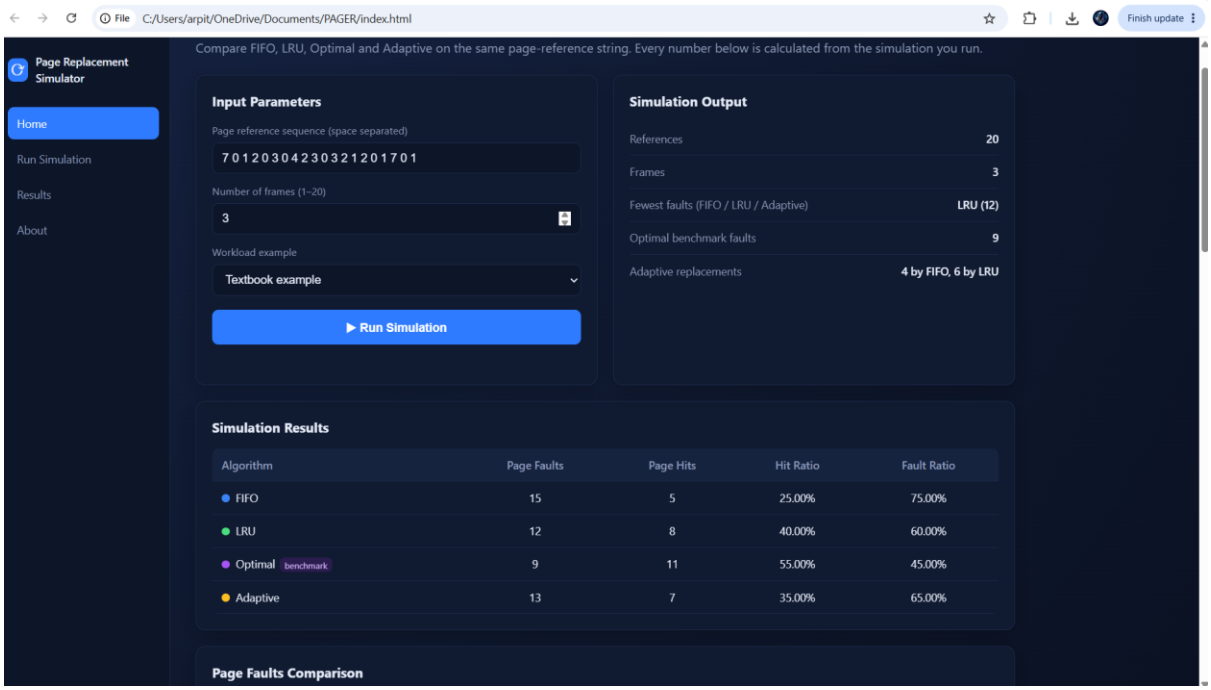
The simulator should be tested with sequential, repeating, textbook, and mixed page-reference sequences. Tests should use different numbers of memory frames to observe changes in algorithm performance.

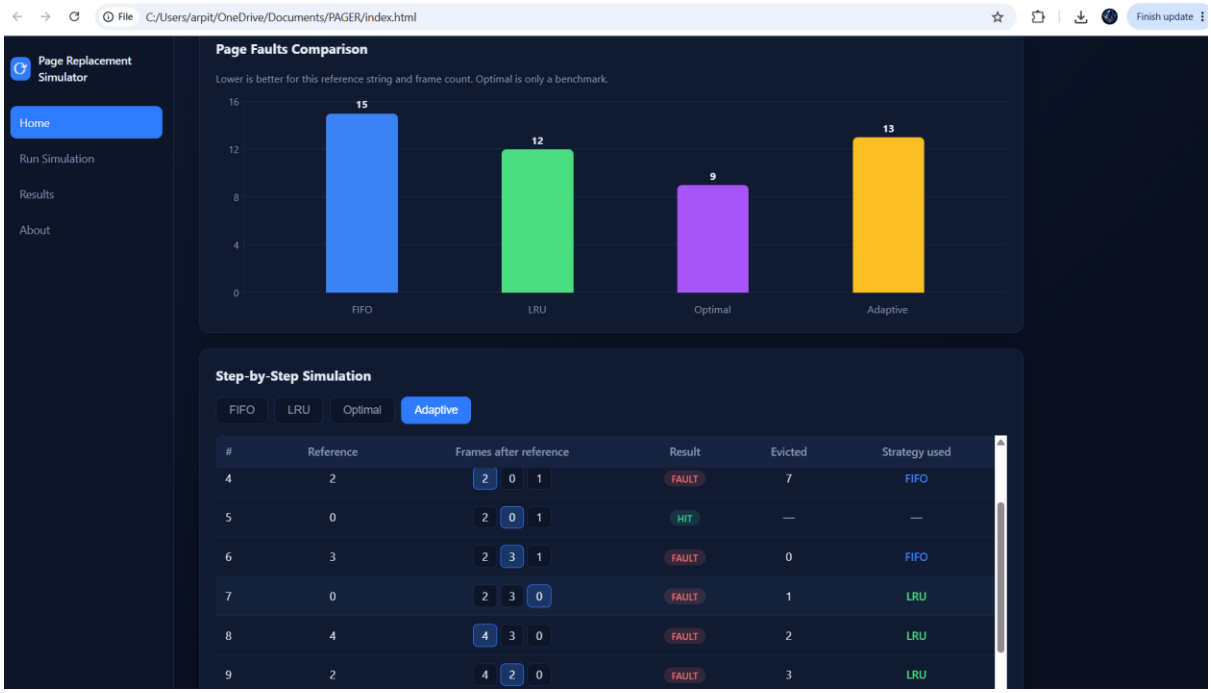
The following checks should be performed:

1. The number of page hits plus page faults must equal the total number of page references.
2. The hit ratio and fault ratio must add up to 1, or 100% when expressed as percentages.
3. The results table and bar chart must update when new inputs are entered.
4. The step-by-step trace must show the correct frame contents and hit or fault status.
5. Adaptive decisions must be displayed when replacements occur.
6. Optimal should produce no more page faults than another correctly implemented algorithm for the same reference sequence and frame count.

8. Result and Screenshots

Textbook Sample Reference:

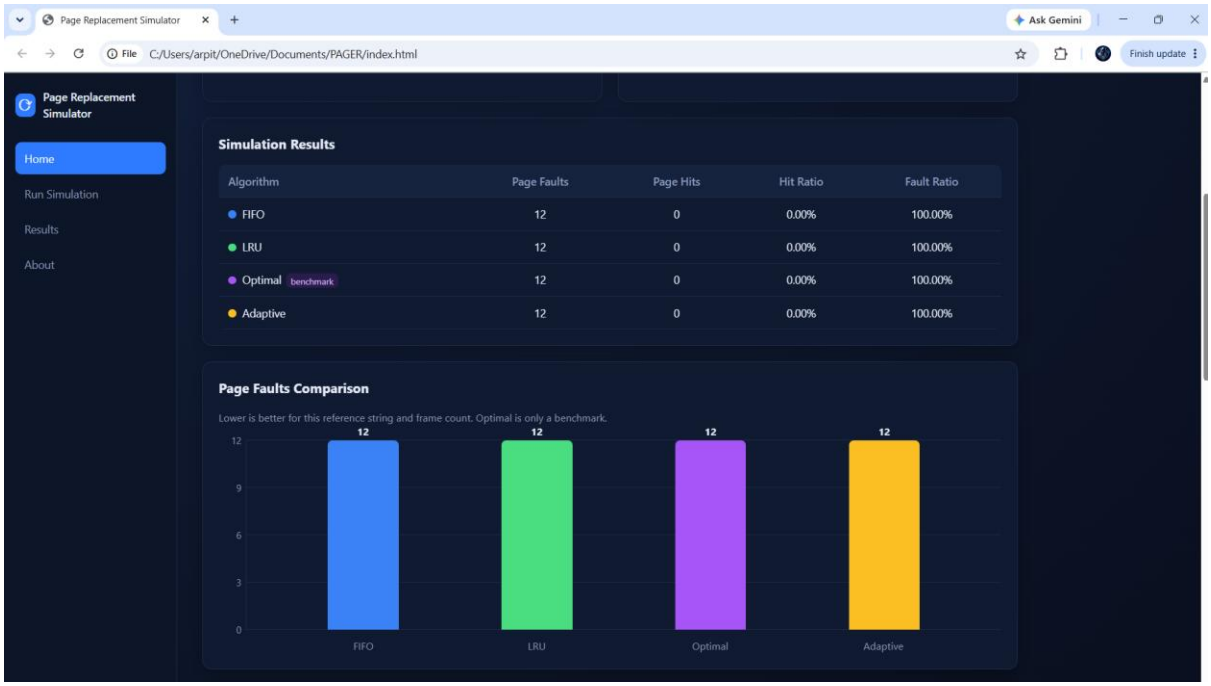


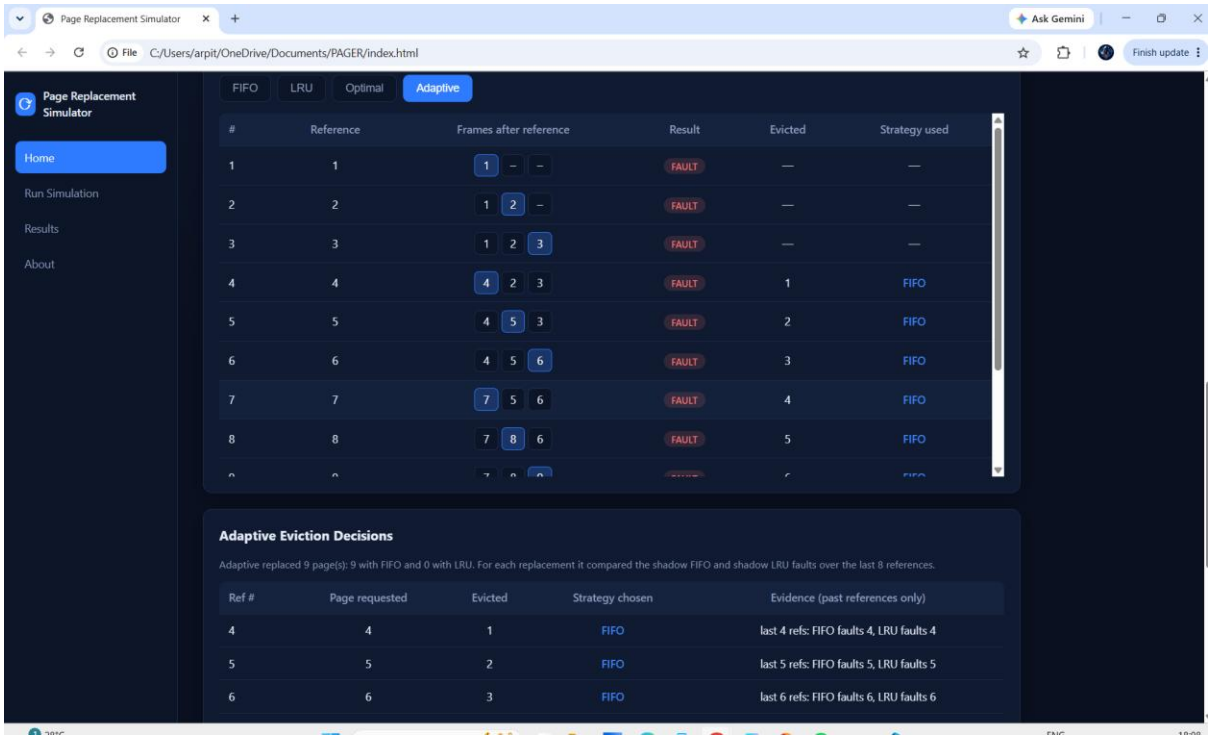




The textbook sample provides a predefined reference sequence for demonstrating page replacement algorithms. It allows users to compare FIFO, LRU, Optimal, and Adaptive under the same input conditions. This test helps illustrate how different replacement strategies select pages for removal and how their performance varies.

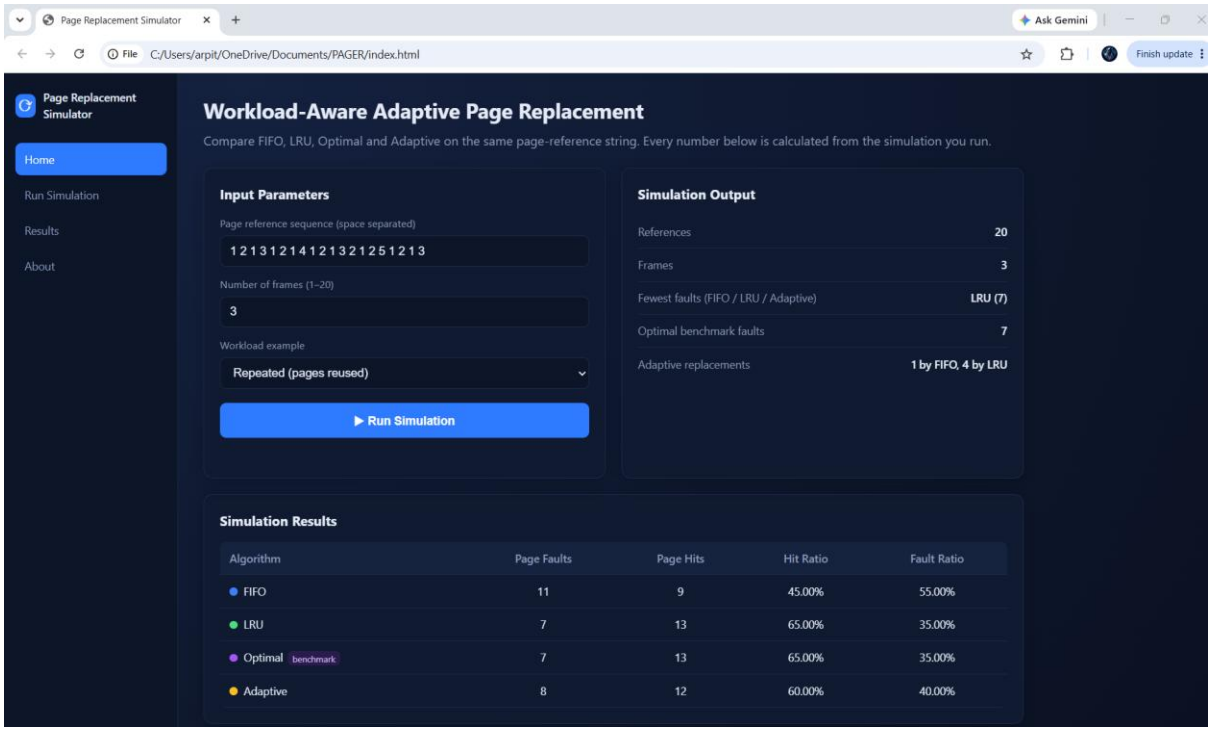
Sequential Reference:

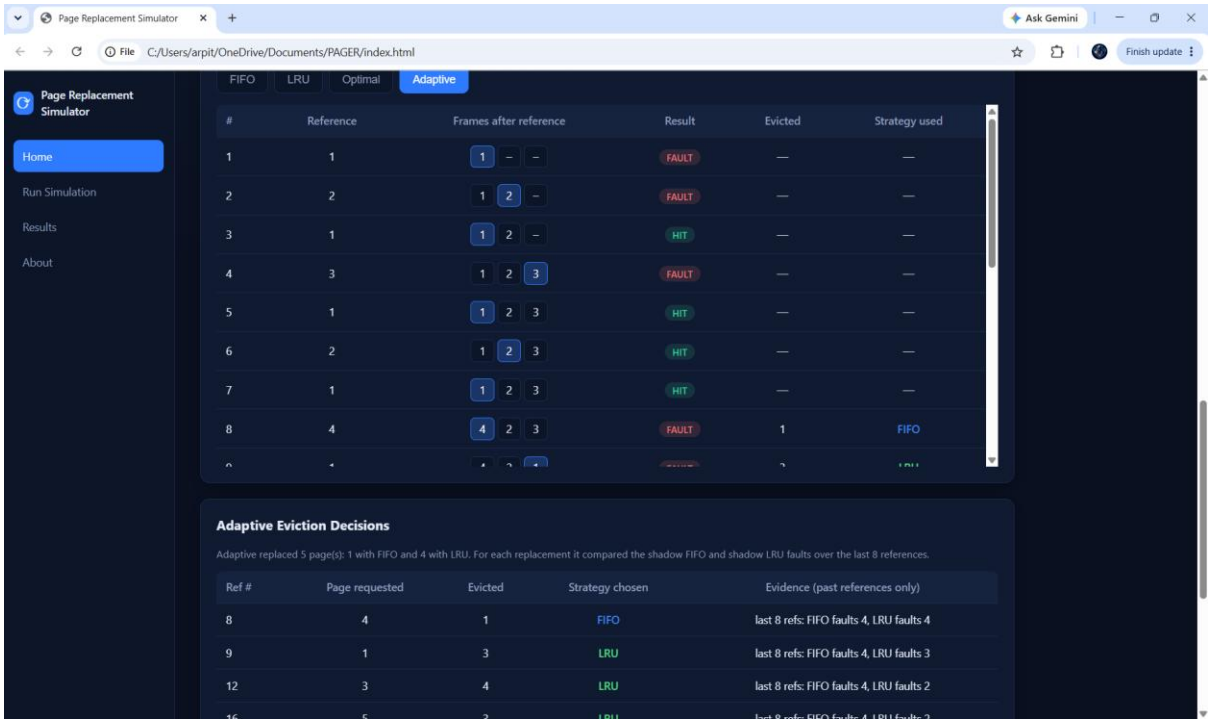




The sequential workload represents page references accessed in a forward sequence without reuse. It evaluates how page replacement algorithms perform when new pages are continuously introduced. Since previously accessed pages are not reused, page faults are high. All four algorithms recorded identical results for this test.

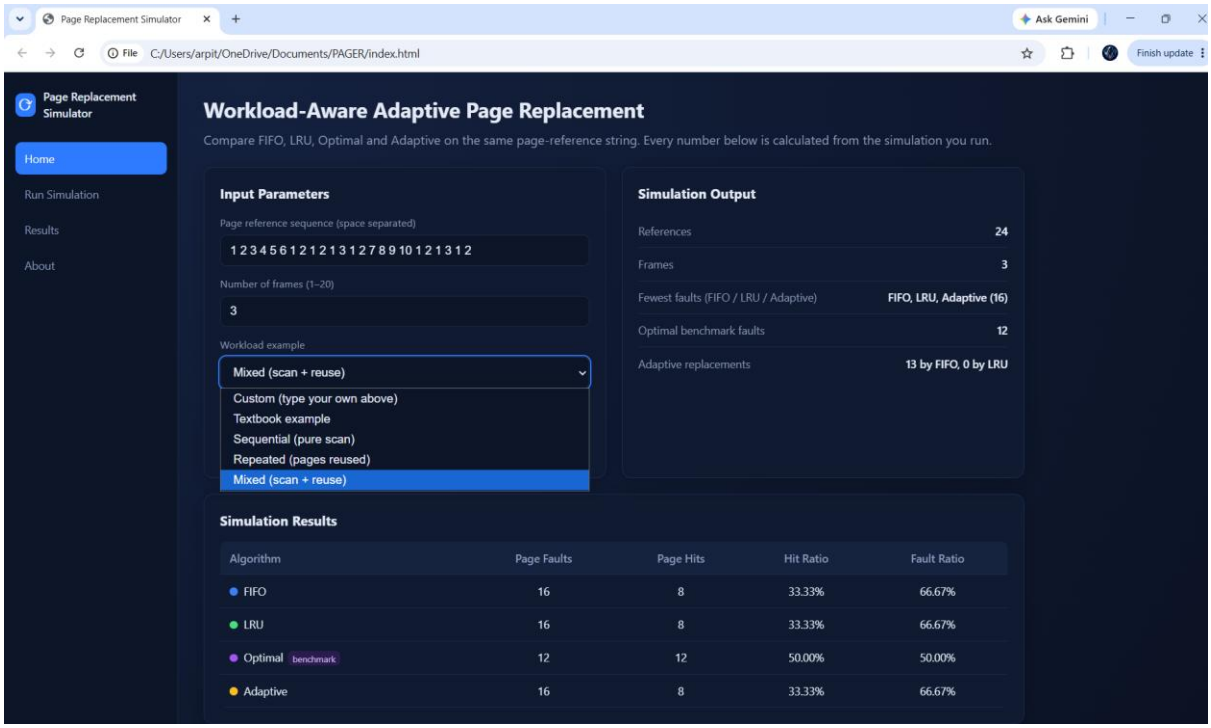
Repeated Reference:

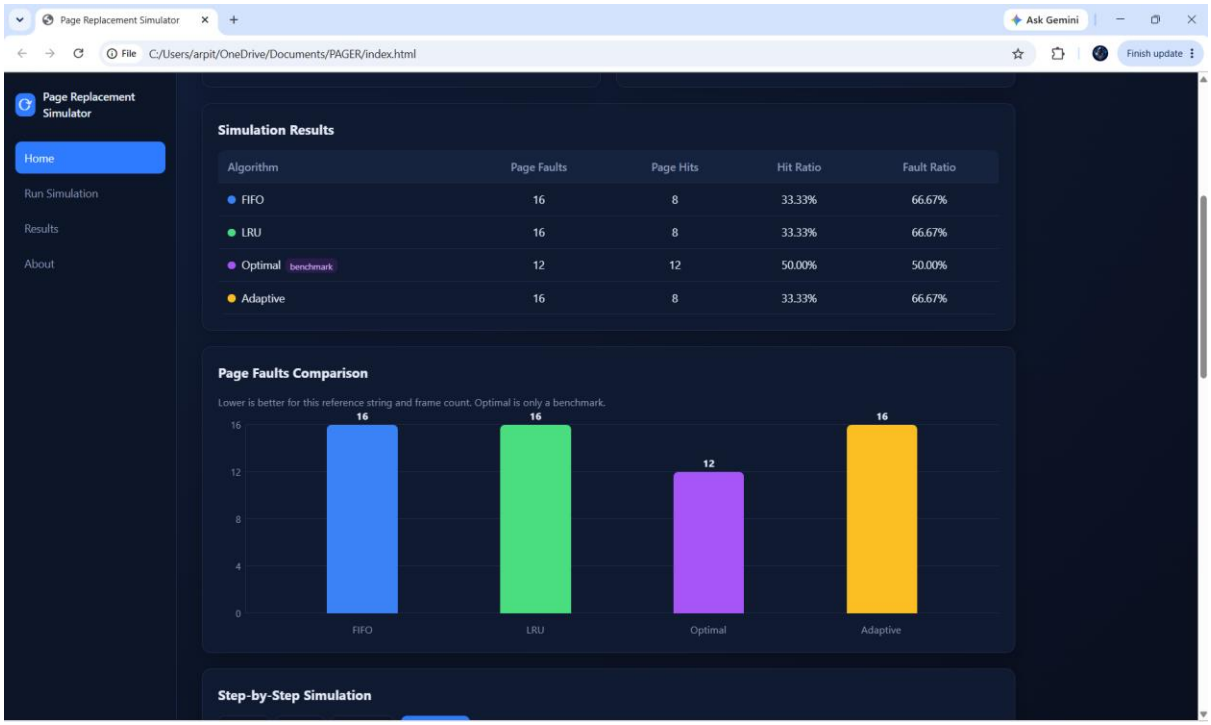




The repeated workload contains frequently reused page references. It evaluates how effectively algorithms retain frequently accessed pages in memory. LRU and Optimal performed better than FIFO in this test. Adaptive also improved upon FIFO by reducing the number of page faults..

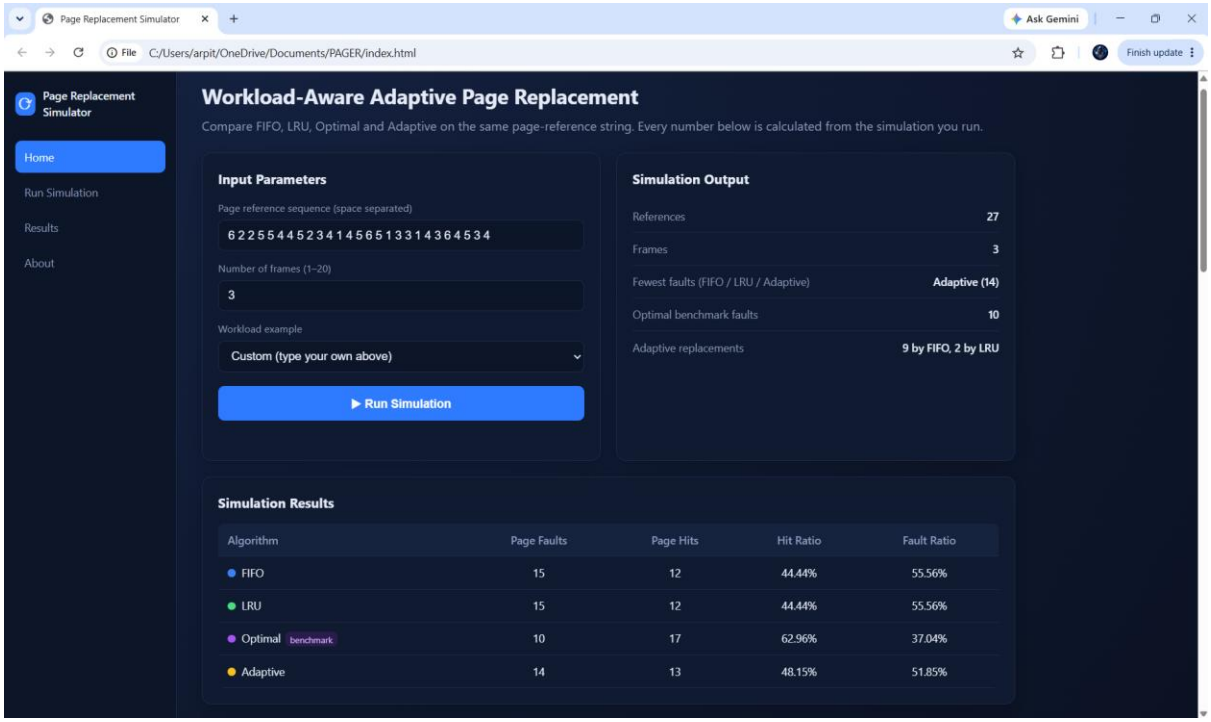
Mixed Reference:

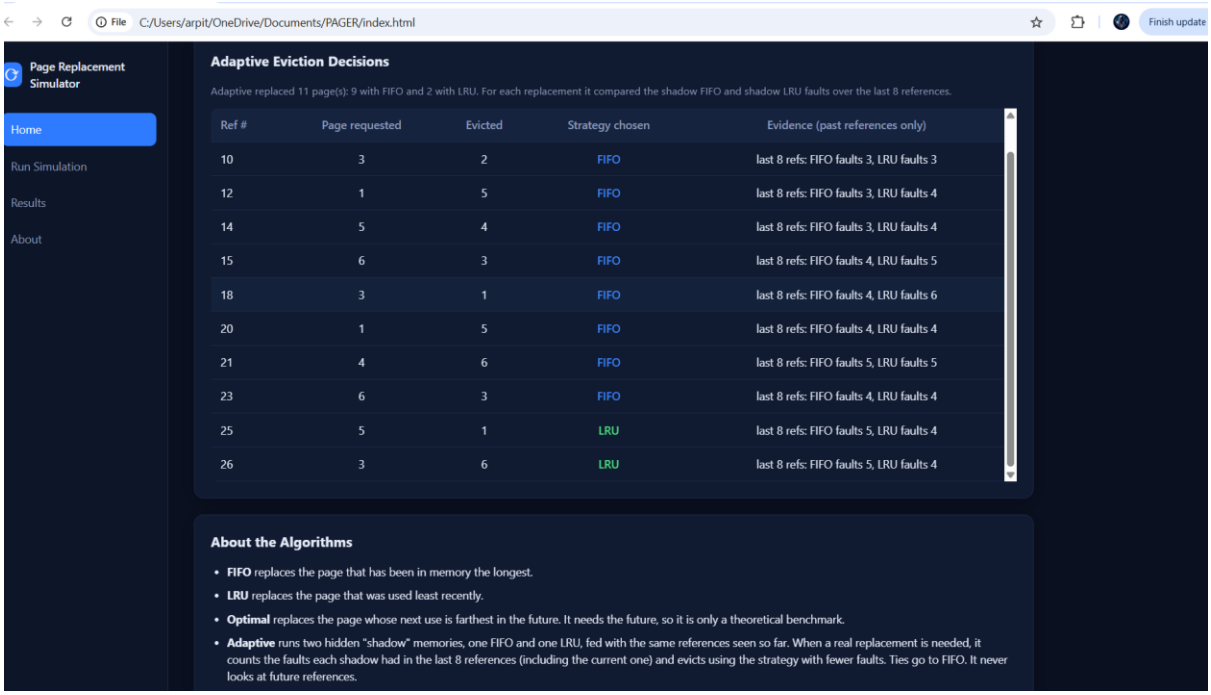




The mixed workload combines new page references with repeated accesses. It helps evaluate algorithm performance under varying access patterns. Optimal recorded fewer page faults than the other algorithms in this test. Adaptive matched the performance of FIFO and LRU.

Custom Reference:



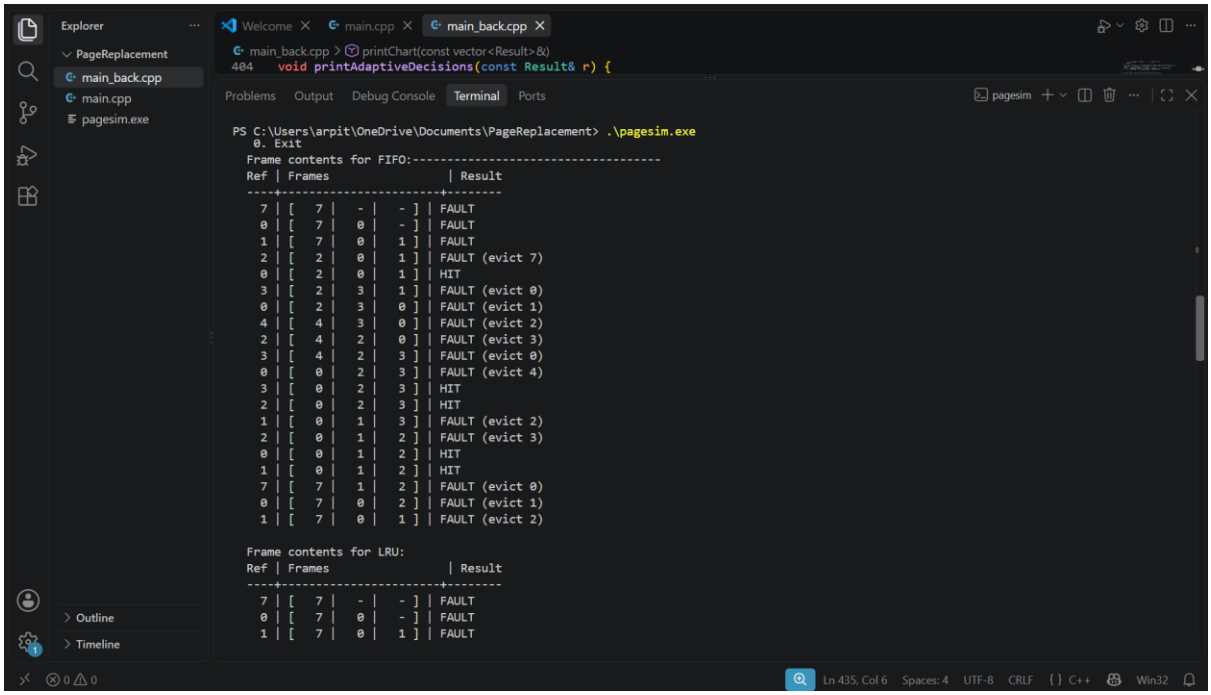


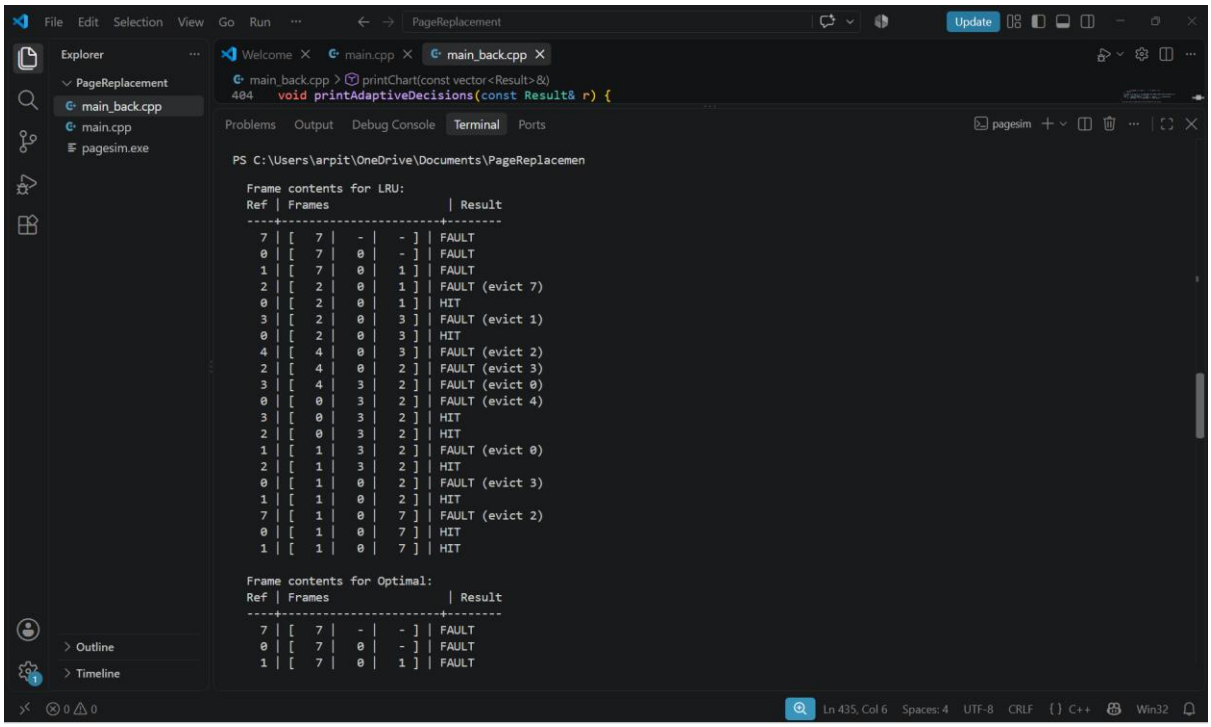
The custom workload allows users to enter their own page-reference sequence and select the number of memory frames. It provides flexibility to test different access patterns. In this test, Optimal recorded the fewest page faults, while Adaptive performed better than FIFO and LRU.

VS Code Simulation:

```
PS C:\Users\arpit\OneDrive\Documents\PageReplacement> g++ -std=c++11 -Wall -Wextra main_back.cpp -o pagesim.exe
PS C:\Users\arpit\OneDrive\Documents\PageReplacement> .\pagesim.exe

============================================================
   Workload-Aware Adaptive Page Replacement Simulator
============================================================
  Reference string : 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1
  Workload type    : Custom (textbook sample)
  Frames           : 3
  Show frame steps : ON
------------------------------------------------------------
   1. Enter custom reference string
   2. Generate workload (sequential / repeated / mixed)
   3. Set number of frames
   4. Toggle frame display ON/OFF
   5. Run FIFO
   6. Run LRU
   7. Run Optimal (theoretical benchmark only)
   8. Run Adaptive (FIFO/LRU)
   9. Compare ALL algorith9s
   0. Exit
```

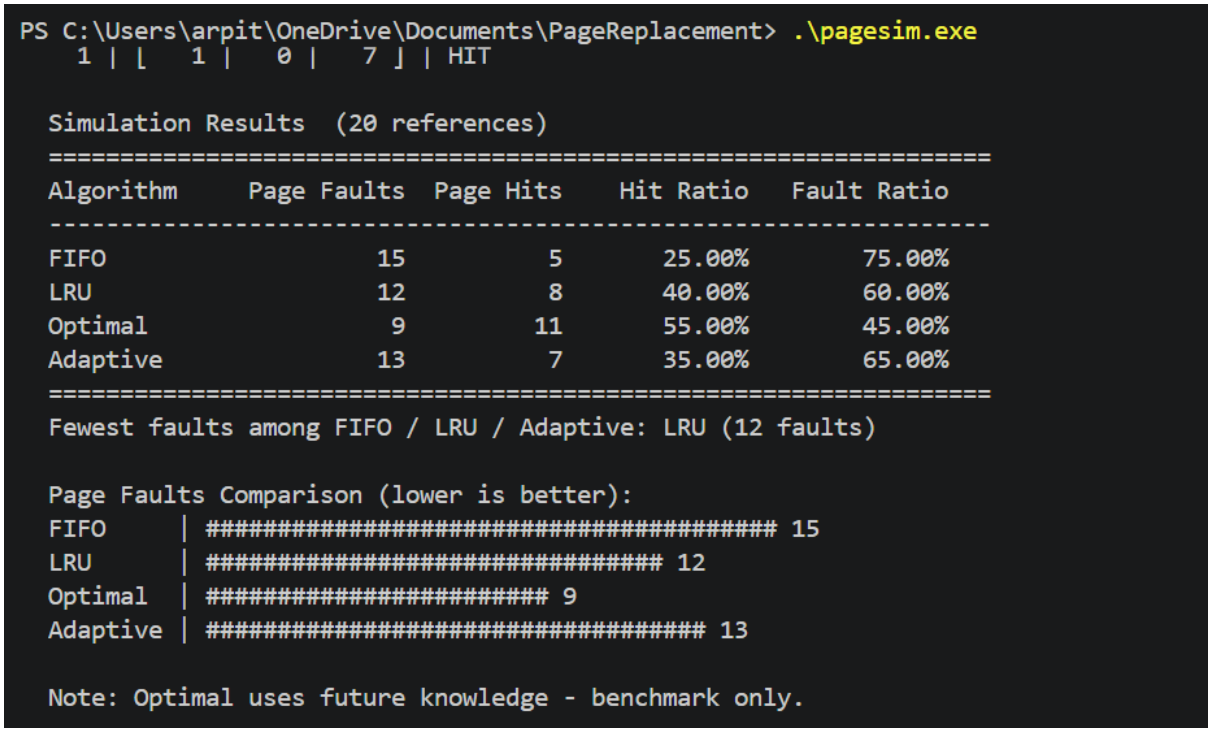

Explorer
PageReplacement
main_back.cpp
main.cpp
pagesim.exe
Welcome
main.cpp
main_back.cpp
main_back.cpp > printChart(const vector<Result>&)
404 void printAdaptiveDecisions(const Result& r) {
Problems
Output
Debug Console
Terminal
Ports
pagesim
PS C:\Users\arpit\OneDrive\Documents\PageReplacement> .\pagesim.exe
0. Exit
Frame contents for FIFO:----------
Ref | Frames | Result
7 | [7 | - | -] | FAULT
0 | [7 | 0 | -] | FAULT
1 | [7 | 0 | 1] | FAULT
2 | [2 | 0 | 1] | FAULT (evict 7)
0 | [2 | 0 | 1] | HIT
3 | [2 | 3 | 1] | FAULT (evict 0)
0 | [2 | 3 | 0] | FAULT (evict 1)
4 | [4 | 3 | 0] | FAULT (evict 2)
2 | [4 | 2 | 0] | FAULT (evict 3)
3 | [4 | 2 | 3] | FAULT (evict 0)
0 | [0 | 2 | 3] | FAULT (evict 4)
3 | [0 | 2 | 3] | HIT
2 | [0 | 2 | 3] | HIT
1 | [0 | 1 | 3] | FAULT (evict 2)
2 | [0 | 1 | 2] | FAULT (evict 3)
0 | [0 | 1 | 2] | HIT
1 | [0 | 1 | 2] | HIT
7 | [7 | 1 | 2] | FAULT (evict 0)
0 | [7 | 0 | 2] | FAULT (evict 1)
1 | [7 | 0 | 1] | FAULT (evict 2)
Frame contents for LRU:
Ref | Frames | Result
7 | [7 | - | -] | FAULT
0 | [7 | 0 | -] | FAULT
1 | [7 | 0 | 1] | FAULT
Outline
Timeline
Ln 435, Col 6
Spaces: 4
UTF-8
CRLF
C++
Win32


File
Edit
Selection
View
Go
Run
PageReplacement
Update
Explorer
PageReplacement
main_back.cpp
main.cpp
pagesim.exe
Welcome
main.cpp
main_back.cpp
main_back.cpp > printChart(const vector<Result>&)
404 void printAdaptiveDecisions(const Result& r) {
Problems
Output
Debug Console
Terminal
Ports
pagesim
PS C:\Users\arpit\OneDrive\Documents\PageReplacemen
Frame contents for LRU:
Ref | Frames | Result
7 | [7 | - | -] | FAULT
0 | [7 | 0 | -] | FAULT
1 | [7 | 0 | 1] | FAULT
2 | [2 | 0 | 1] | FAULT (evict 7)
0 | [2 | 0 | 1] | HIT
3 | [2 | 0 | 3] | FAULT (evict 1)
0 | [2 | 0 | 3] | HIT
4 | [4 | 0 | 3] | FAULT (evict 2)
2 | [4 | 0 | 2] | FAULT (evict 3)
3 | [4 | 3 | 2] | FAULT (evict 0)
0 | [0 | 3 | 2] | FAULT (evict 4)
3 | [0 | 3 | 2] | HIT
2 | [0 | 3 | 2] | HIT
1 | [1 | 3 | 2] | FAULT (evict 0)
2 | [1 | 3 | 2] | HIT
0 | [1 | 0 | 2] | FAULT (evict 3)
1 | [1 | 0 | 2] | HIT
7 | [1 | 0 | 7] | FAULT (evict 2)
0 | [1 | 0 | 7] | HIT
1 | [1 | 0 | 7] | HIT
Frame contents for Optimal:
Ref | Frames | Result
7 | [7 | - | -] | FAULT
0 | [7 | 0 | -] | FAULT
1 | [7 | 0 | 1] | FAULT
Outline
Timeline
Ln 435, Col 6
Spaces: 4
UTF-8
CRLF
C++
Win32

```
Frame contents for Optimal:
Ref | Frames                  | Result
----+-------------------------+--------
  7 | [   7 |   - |   - ] | FAULT
  0 | [   7 |   0 |   - ] | FAULT
  1 | [   7 |   0 |   1 ] | FAULT
  2 | [   2 |   0 |   1 ] | FAULT (evict 7)
  0 | [   2 |   0 |   1 ] | HIT
  3 | [   2 |   0 |   3 ] | FAULT (evict 1)
  0 | [   2 |   0 |   3 ] | HIT
  4 | [   2 |   4 |   3 ] | FAULT (evict 0)
  2 | [   2 |   4 |   3 ] | HIT
  3 | [   2 |   4 |   3 ] | HIT
  0 | [   2 |   0 |   3 ] | FAULT (evict 4)
  3 | [   2 |   0 |   3 ] | HIT
  2 | [   2 |   0 |   3 ] | HIT
  1 | [   2 |   0 |   1 ] | FAULT (evict 3)
  2 | [   2 |   0 |   1 ] | HIT
  0 | [   2 |   0 |   1 ] | HIT
  1 | [   2 |   0 |   1 ] | HIT
  7 | [   7 |   0 |   1 ] | FAULT (evict 2)
  0 | [   7 |   0 |   1 ] | HIT
  1 | [   7 |   0 |   1 ] | HIT
```

```
PS C:\Users\arpit\OneDrive\Documents\PageReplacement> .\pagesim.exe

  Frame contents for Adaptive:
  Ref | Frames                  | Result
  ----+-------------------------+--------
    7 | [   7 |   - |   - ] | FAULT
    0 | [   7 |   0 |   - ] | FAULT
    1 | [   7 |   0 |   1 ] | FAULT
    2 | [   2 |   0 |   1 ] | FAULT (evict 7) [strategy: FIFO]
    0 | [   2 |   0 |   1 ] | HIT
    3 | [   2 |   3 |   1 ] | FAULT (evict 0) [strategy: FIFO]
    0 | [   2 |   3 |   0 ] | FAULT (evict 1) [strategy: LRU]
    4 | [   4 |   3 |   0 ] | FAULT (evict 2) [strategy: LRU]
    2 | [   4 |   2 |   0 ] | FAULT (evict 3) [strategy: LRU]
    3 | [   4 |   2 |   3 ] | FAULT (evict 0) [strategy: LRU]
    0 | [   0 |   2 |   3 ] | FAULT (evict 4) [strategy: LRU]
    3 | [   0 |   2 |   3 ] | HIT
    2 | [   0 |   2 |   3 ] | HIT
    1 | [   1 |   2 |   3 ] | FAULT (evict 0) [strategy: LRU]
    2 | [   1 |   2 |   3 ] | HIT
    0 | [   1 |   0 |   3 ] | FAULT (evict 2) [strategy: FIFO]
    1 | [   1 |   0 |   3 ] | HIT
    7 | [   1 |   0 |   7 ] | FAULT (evict 3) [strategy: FIFO]
    0 | [   1 |   0 |   7 ] | HIT
    1 | [   1 |   0 |   7 ] | HIT
```

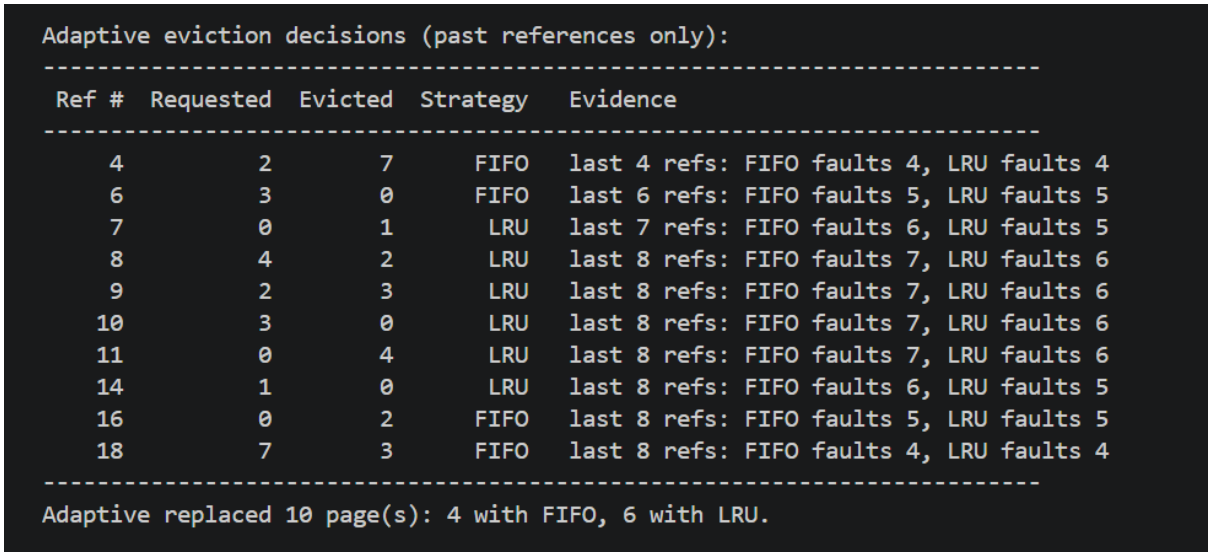


```
PS C:\Users\arpit\OneDrive\Documents\PageReplacement> .\pagesim.exe
    1 | [   1 |   0 |   7 ] | HIT

  Simulation Results  (20 references)
  ====================================================================
  Algorithm     Page Faults  Page Hits    Hit Ratio   Fault Ratio
  --------------------------------------------------------------------
  FIFO                   15          5       25.00%        75.00%
  LRU                    12          8       40.00%        60.00%
  Optimal                 9         11       55.00%        45.00%
  Adaptive               13          7       35.00%        65.00%
  ====================================================================
  Fewest faults among FIFO / LRU / Adaptive: LRU (12 faults)

  Page Faults Comparison (lower is better):
  FIFO     | ######################################## 15
  LRU      | ################################## 12
  Optimal  | ######################### 9
  Adaptive | ##################################### 13

  Note: Optimal uses future knowledge - benchmark only.
```



```
  Adaptive eviction decisions (past references only):
  ------------------------------------------------------------------------
   Ref #  Requested  Evicted  Strategy   Evidence
  ------------------------------------------------------------------------
      4          2        7      FIFO   last 4 refs: FIFO faults 4, LRU faults 4
      6          3        0      FIFO   last 6 refs: FIFO faults 5, LRU faults 5
      7          0        1       LRU   last 7 refs: FIFO faults 6, LRU faults 5
      8          4        2       LRU   last 8 refs: FIFO faults 7, LRU faults 6
      9          2        3       LRU   last 8 refs: FIFO faults 7, LRU faults 6
     10          3        0       LRU   last 8 refs: FIFO faults 7, LRU faults 6
     11          0        4       LRU   last 8 refs: FIFO faults 7, LRU faults 6
     14          1        0       LRU   last 8 refs: FIFO faults 6, LRU faults 5
     16          0        2      FIFO   last 8 refs: FIFO faults 5, LRU faults 5
     18          7        3      FIFO   last 8 refs: FIFO faults 4, LRU faults 4
  ------------------------------------------------------------------------
  Adaptive replaced 10 page(s): 4 with FIFO, 6 with LRU.
```

This set of screenshots illustrates the execution of the Workload-Aware Adaptive Page Replacement Simulator in VS Code. The command-line interface allows users to enter a custom page-reference string, generate sequential, repeated, or mixed workloads, configure the number of memory frames, and enable or disable the display of frame-by-frame steps. The simulator implements four page replacement algorithms: FIFO, LRU, Optimal, and Adaptive. It displays the page replacement process, including page hits, page faults, frame updates, and performance comparisons between algorithms. The results include metrics such as total page faults, page hits, hit ratio, and fault ratio, allowing users to evaluate algorithm performance under different

workloads. The Adaptive algorithm additionally displays its eviction decisions and the evidence used to select FIFO or LRU based on recent page-fault observations. Together, these screenshots demonstrate the simulator's functionality, algorithm execution, workload handling, and performance analysis through a terminal-based interface developed in C++.

9. GitHub Repository and Submission

GitHub Repository Link: [https://github.com/arpitraj702-del/Workload-Aware-Adaptive-Page-Replacement.git]

10. Results and Discussion

The project was evaluated using four page-reference workloads: sequential, repeated, mixed, and custom. The FIFO, LRU, Optimal, and Adaptive algorithms were compared using page faults, page hits, hit ratio, and fault ratio. Each website simulation used three memory frames.

10.1 Sequential Workload

In the sequential workload, all four algorithms recorded 12 page faults and zero page hits. The hit ratio was 0%, while the fault ratio was 100%. This occurred because the workload did not provide opportunities to reuse previously loaded pages.

Reference string: 1 2 3 4 5 6 7 8 9 10 11 12

Algorithm	Page Faults	Page Hits	Hit Ratio	Fault Ratio
FIFO	12	0	0.00%	100.00%
LRU	12	0	0.00%	100.00%
Optimal	12	0	0.00%	100.00%
Adaptive	12	0	0.00%	100.00%

10.2 Repeated Workload

In the repeated workload, LRU and Optimal achieved the best results, with seven page faults and thirteen page hits each. Adaptive recorded eight page faults, performing better than FIFO, which recorded eleven. These results demonstrate how repeated page accesses can benefit from replacement strategies that retain useful pages in memory.

Reference string: 1 2 1 3 1 2 1 4 1 2 1 3 2 1 2 5 1 2 1 3

Algorithm	Page Faults	Page Hits	Hit Ratio	Fault Ratio
FIFO	11	9	45.00%	55.00%
LRU	7	13	65.00%	35.00%
Optimal	7	13	65.00%	35.00%
Adaptive	8	12	60.00%	40.00%

10.3 Mixed Workload

For the mixed workload, FIFO, LRU, and Adaptive each recorded sixteen page faults and eight page hits. Optimal performed better, recording twelve page faults and twelve page hits. The results show that a workload containing both new and repeated references can produce different outcomes across replacement algorithms.

Reference string: 1 2 3 4 5 6 1 2 1 3 1 2 7 8 9 10 1 2 3 1 2 1 3 12

Algorithm	Page Faults	Page Hits	Hit Ratio	Fault Ratio
FIFO	16	8	33.33%	66.67%
LRU	16	8	33.33%	66.67%
Optimal	12	12	50.00%	50.00%
Adaptive	16	8	33.33%	66.67%

10.4 Custom Workload

The custom workload allowed a user-defined page-reference sequence to be tested. Optimal recorded the fewest page faults, with ten, while Adaptive recorded fourteen. FIFO and LRU each recorded fifteen page faults. For this input, Adaptive performed slightly better than FIFO and LRU but did not match the Optimal benchmark.

Reference string: 6 2 2 5 5 4 4 5 2 3 4 1 4 5 6 5 1 3 3 1 4 3 6 4 5 3 4

Algorithm	Page Faults	Page Hits	Hit Ratio	Fault Ratio
FIFO	15	12	44.44%	55.56%
LRU	15	12	44.44%	55.56%
Optimal	10	17	62.96%	37.04%
Adaptive	14	13	48.15%	51.85%

10.5 Overall Discussion

The results indicate that page replacement performance depends on the page-reference sequence and the opportunities for page reuse. FIFO uses the order in which pages enter memory, while LRU uses recent access history. Optimal serves as a theoretical benchmark because it uses future-reference knowledge to minimize page faults for a given sequence and frame count.

The Adaptive algorithm compares the recent page-fault performance of FIFO and LRU using its observation window. It selects the policy with fewer observed faults, choosing FIFO when the results are tied. This approach allows it to respond to recent workload behavior, but it does not guarantee fewer page faults than both FIFO and LRU for every input.

The website screenshots provide evidence of the simulations and the performance metrics displayed for each workload. The tables summarize these measurements and make it easier to compare the algorithms under different access patterns.

The project demonstrates the relationship between workload characteristics and page replacement performance. It also illustrates how an adaptive strategy can use observed results to guide replacement decisions. The VS Code

implementation and its test results will provide an additional view of the project once their outputs are included.

11. Conclusion

The Workload-Aware Adaptive Page Replacement project demonstrates the operation of common page replacement algorithms and compares their performance using page faults, page hits, and related metrics. The web interface provides a visual and interactive way to understand memory replacement decisions. The Adaptive strategy illustrates how recent workload behaviour can guide the selection between FIFO and LRU, while Optimal serves as a theoretical benchmark for comparison.

The project combines operating system concepts with C++ programming and web technologies to provide an educational tool for understanding virtual memory management.